

MEAM 6200 - Project 3

Sonam Okuda

I. INTRODUCTION

This report details the design and implementation of autonomous quadcopter systems, where the primary objective was to develop components required for UAV flight, including feedback control, motion planning, trajectory generation, and state estimation. The end goal was to allow quadcopters to navigate complex three-dimensional environments while maintaining stability, avoiding collisions, and optimizing flight performance.

II. GENERAL APPROACH

A. Controller

Firstly, the attitude and position controller was developed. Using a geometry-based approach to address the non-linearity of the system, an SE(3) controller was tuned and implemented; the controller took in the desired flat output, which consisted of

$$\underline{x} = [x, \dot{x}, \ddot{x}, \ddot{\ddot{x}}, \ddot{\ddot{\ddot{x}}}, \psi, \dot{\psi}].$$

From the equations of motion, the linear and angular velocities can be determined by specifying the thrust and moment, which are the components of the controller output, $\underline{u} = [u_1, u_2]$.

The controller gains, K_p , K_d , K_r , and K_w are diagonal matrices that apply proportional gain and damping to the linear and angular state errors, respectively.

B. A* Implementation

To generate a set of waypoints for trajectory generation, the A* algorithm was used to find the optimal path through a 3D voxel occupancy grid. The algorithm searched nearby neighbors and calculated the cost and heuristic, the Euclidean distance to the goal. However, because the A* path consisted of densely packed nodes where the trajectory had to navigate complex or narrow geometry, an LOS pruner was implemented; all nodes in between nodes A and B were pruned, if A and B maintained complete line-of-sight. By increasing the safety margin to 0.3, the LOS pruner, while aggressive still guaranteed the segment to be collision-free. In order to allow the A* search to effectively, particularly through maps with windows or gaps, the resolution was set to 0.25.

C. Trajectory Generation

After generating waypoints, a quintic spline was fitted between each waypoint to ensure that the flight was continuous and smooth. By setting the accelerations at each waypoint to zero and ensuring that the end velocity of the previous segment matched the start velocity of the following segment, a smooth quintic spline could be fitted between each waypoint by solving for the quintic coefficients.

D. Stereo Odometry

To estimate the pose of a quadcopter, stereo camera correspondences can be used to find the rotation and translation. A least-squares solve over normalized stereo measurements recovers the angular velocity vector and translation by linearizing the rigid body motion between frames. To reduce the number of outlier correspondences, the RANSAC algorithm was used to identify inliers by repeatedly fitting the model to the small random subsets of correspondences. The algorithm then evaluates the model based on how many inliers are consistent with the model, and after identifying the best candidate, it discards outliers. This process repeats until RANSAC converges on a rotation and translation.

E. State Estimation

The primary method used for quadcopter state estimation was Visual-Inertial Odometry (VIO), implemented using an Error-State Kalman Filter (ESKF). The ESKF maintained a nominal state (position, velocity, orientation quaternion, accelerometer bias, gyroscope bias, and gravity estimate). Between visual observations, the nominal state was propagated forward by integrating raw IMU measurements while correcting for current bias estimates, and the error-state covariances were recalculated using the linearized process Jacobian and process noise contributions. The stereo odometry observations were then used to correct the IMU propagation; by calculating the Kalman gain, the innovation can then be added to form a better estimate of the quadcopter's actual position and attitude by combining the high-frequency IMU data with the drift-correction from the stereo measurements.

III. ADJUSTMENTS

A. Controller Gain and Map Resolution Tuning

After implementing the VIO, the quadcopter experienced significant drift; the controller was unable to

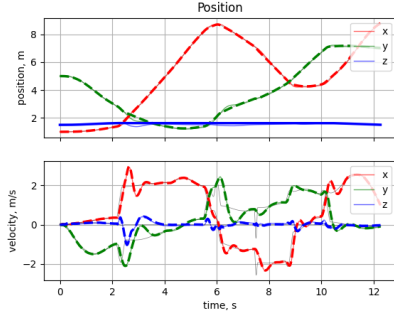


Fig. 1: Controller Performance

keep up with the velocities required by the generated trajectory, allowing the quadcopter to encounter collisions more frequently. In order to compensate for the additional drift, I increased the controller gains in the x-direction only, as increasing gains in both directions made the controller extremely unstable. Usually, making the controller more aggressive in an asymmetric fashion could cause severe lateral drift; however, because that the quadrotor has to fly generally in the x-direction for the obstacle maps, I was willing to risk the tradeoff, which became apparent in the suboptimal performance of the quadrotor in the slalom and switchback maps. Maps such as the over-under and stairwell performed well, which implied that the tracking in the z-direction did not need additional controller boosts.

B. Clearance Penalty

Another issue I encountered was that reducing the map resolution encouraged the A* algorithm to hug obstacles; coupled with my struggling controller, this resulted in increased collisions. To discourage approaching obstacles, I implemented a clearance penalty which I added to the edge cost when calculating the heuristics for A*.

C. Dynamic Speed Allocation for Sharp Turns

To help my controller but maintain a fast trajectory, I implemented scale factors that decreased speed if the angle subtended by two adjacent segments was larger than the threshold angle of 10 degrees. I further decreased the speed of the trajectory required a particularly sharp turn (threshold angle of 30 degrees). If the splines between the segments were relatively straight (less than ten degrees), I increased the segment velocity, which helped reduce flight time for several maps.

IV. MAZE FLIGHT EXPERIMENTS

To test my integration, I created window, over-under, slalom, and maze maps. The following images illustrate the drift and obstacle hugging issues that I encountered. Figure 2 illustrates maps my system already performed

well on, with little deviation in the trajectory tracking and optimal trajectory generation. In the slalom performance, the old trajectory demanded a high velocity when rounding the slalom walls; the controller struggled to track precisely, resulting in a bulge that caused a collision. To combat this, the dynamic speed allocation was implemented, and as the obstacle avoidance segments had high turn angles, the speed was decreased, allowing the trajectory to be tracked more accurately. The maze implementation illustrates the clearance penalty: by penalizing nodes near obstacles and setting a high margin on 0.3, A* completely avoided narrow passageways in the map. While this trajectory is suboptimal, it is much safer than squeezing through the passages. I made this design decision to prioritize collision avoidance over speed and optimality.

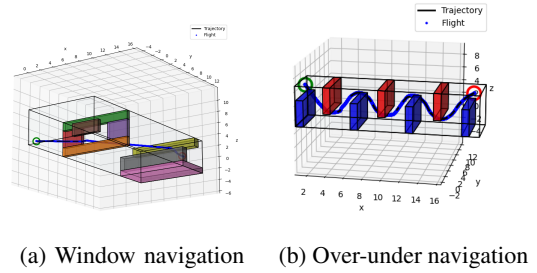


Fig. 2: Good Performance Maps

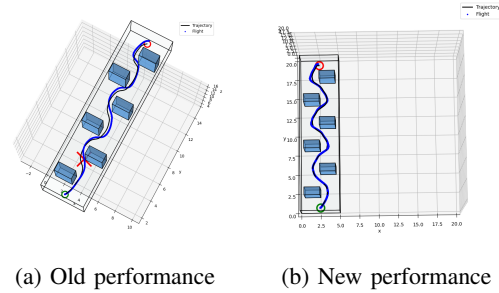


Fig. 3: Slalom Performance

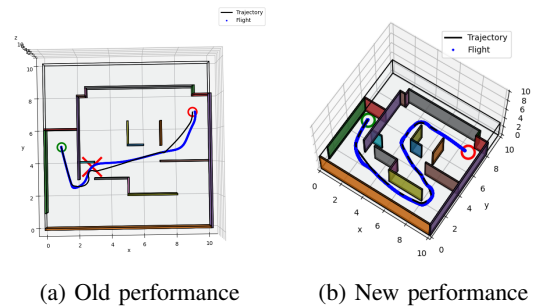


Fig. 4: Maze Performance